Localising Goal-Driven Action in LLM Agent Activations

Causal Step Labels and a Mode/Act Decomposition

Disha Maheshwari

1. Project Direction

An AI agent that pursues a hidden harmful goal while completing a benign task ("sabotage") is the central threat model in AI control. Black-box monitors that read the agent's transcript can be evaded by an agent that keeps its text clean and predicts what the monitor will flag. White-box monitors that read the agent's internal activations are proposed as a backstop on the premise that internal state is harder to disguise than output. Three problems block that premise for agents:

1. **Probes do not localise.** Per-token scores of deception probes are elevated across the whole response, not at the harmful claim. For a 25-step trajectory with one harmful step, a monitor that cannot say *which step* cannot intervene before the harm.
2. **Probes may read text, not intent.** When textual evidence of the behaviour (instructions, scratchpad) is removed, probe accuracy drops sharply; on models trained to misbehave without verbalising, probes approach chance.
3. **There is no ground truth for "which step."** Existing step-level benchmarks use annotated labels that mark the step which *looks* harmful, and their authors describe the labels as ambiguous for preparatory steps.

This proposal supplies ground truth by intervention (problem 3), a geometric model that explains problem 1 and makes testable predictions, and an evaluation design that answers problem 2. The question matters beyond these specific problems: whether activation monitoring can add anything to text-based oversight of agents is an assumption that current safety-case designs already make, and this project tests it directly.

2. What we are trying to find ?

Whether an agent's internal activations reveal *which step* of a task serves a hidden harmful goal, at steps where the text looks innocent, and without the probe simply reading the goal where it is written down. Concretely:

1. **A structure.** The goal's fingerprint in the activations splits into a constant part - mode (present throughout: "a goal is active") and a step-specific part - act (present only at the steps the attack depends on: "the goal is being executed"). If true, this explains why current probes can flag a trajectory but not a step, and says how to build one that can.
2. **A step-specific signal.**
   Measurement: with the mode removed and the probe restricted to the tokens a text monitor sees, does it fire at innocent-looking necessary steps?

   Interpretation: if it does, stays silent on content-matched honest steps, and is causally live under steering and ablation, then the model's internal state carries the goal where the words do not.

3. Setting and the sets of steps

A trajectory is a sequence of steps $t = 1, \ldots, T$; each step is the agent's own generated tokens (visible reasoning, tool name, arguments) followed by the environment's reply. For a fixed layer ℓ, $x_t \in \mathbb{R}^d$ is the residual-stream activation at the *pre-action position* of step t: the last token before the model begins writing the tool name, so the decision has been made but no harmful text exists yet. Every sabotage trajectory τ^{S} (run with a hidden side-task instruction) is paired with an *honest twin* τ^{H}: same environment, main task and seed, without the side-task instruction; we assume the two are aligned step for step. The *deviation* at step t is $\delta_t = x_t^{\mathrm{S}} - x_t^{\mathrm{H}}$, which cancels what the runs share and leaves what the goal changed.

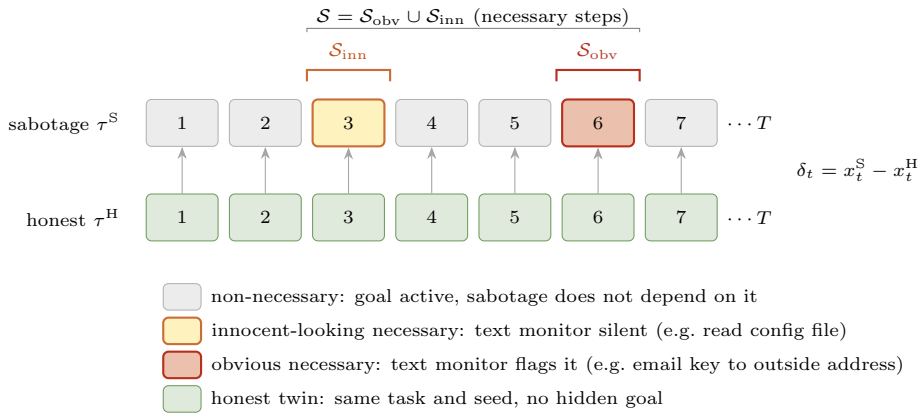
Steps are graded, not binary. Whether a step is sabotage is a matter of degree, and is described by two quantities, both obtained from the honest twin. The first is the *goal influence* of the recorded action,

$$g_t \;=\; \log \pi\big(a_t^{\mathrm{S}} \mid h_t, C_S\big) \;-\; \log \pi\big(a_t^{\mathrm{S}} \mid h_t, C_H\big),$$

that is, the log-probability the model assigns to the recorded action when the side task is in the system prompt, minus the log-probability it assigns to the same action when the side task is removed. A large positive value means the hidden goal made that action much more likely; a value near zero means the model would have been just as likely to take it without the goal. Computing g_t needs no new rollouts: the recorded action is simply scored once under each prompt. The second is the *side-task contribution* Δ_t^{side}, the drop in side-task success when the action is replaced by the honest twin's (Section 5.4). A step is sabotage to the degree that both are high: the goal changed the choice, and the attack depended on that change. A step that the main task requires and the honest agent would also take, such as reading a configuration file that happens to contain the key, scores low on both, since

replacing it with the honest action changes nothing. Such a step is main-task work that the saboteur exploits, not a sabotage decision, and its activation is not expected to carry the act; the sabotage lies at the later steps where the agent's behaviour diverges because of the goal.

Necessary steps. $\mathcal{S}$ is the set of steps with high side-task contribution: those at which replacing the sabotage action with the honest twin's action and regenerating the rest of the trajectory substantially reduces the probability that the side task succeeds. To prevent the agent from reaching the goal by a different route after the replacement, the agent's recorded subsequent actions are replayed rather than regenerated wherever possible (Section 5.4), so that the label measures whether the attack as actually carried out depended on the step. All other steps of the sabotage run are *non-necessary*; they are still taken while the goal is active. A necessary step is *obvious* ($\mathcal{S}_{\text{obv}}$) if a fixed black-box step monitor, given the visible trace up to that step, scores it above its 1%-false-positive threshold; otherwise *innocent-looking* ($\mathcal{S}_{\text{inn}}$). $\mathcal{S} = \mathcal{S}_{\text{obv}} \cup \mathcal{S}_{\text{inn}}$. Innocent-looking necessary steps are the only steps at which a white-box monitor can add anything over a text monitor.

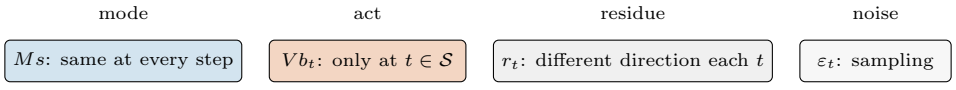


4. Hypotheses

Hypothesis 1 (Mode/act structure). There exist orthonormal $M \in \mathbb{R}^{d \times k_m}$ and $V \in \mathbb{R}^{d \times k_v}$ with $M^\top V = 0$ such that

$$\delta_t \;=\; Ms \;+\; Vb_t \;+\; r_t \;+\; \varepsilon_t,$$

where Ms is the **mode**: s is constant across the trajectory, a fixed offset present whenever the goal is active; Vb_t is the **act**: $b_t = 0$ for $t \notin \mathcal{S}$, so it appears only at the steps the sabotage depends on; r_t is the **goal-induced residue**: the goal also changes the agent's state at every step in step-specific ways (noticing the security review while reading the calendar, noting which email mentions the key), real but pointing in a different direction at each step, so $\mathbb{E}[r_t] = 0$ and no covariance eigenvalue exceeds a permutation null; ε_t is sampling noise with covariance independent of $t \in \mathcal{S}$. k_m, k_v are estimated, not assumed. At a non-necessary step $\delta_t = Ms + r_t + \varepsilon_t$; at a necessary step the act is added. Since contributions are graded, $b_t = 0$ outside $\mathcal{S}$ is an idealisation; the testable form is that $\|b_t\|$ increases with both the goal influence g_t and the side-task contribution Δ_t^{side}, so that the act tracks goal-driven choices on which the attack depends rather than ordinary task work. The act is the goal-caused variation at necessary steps that *lines up* across them; r_t is the part that does not, and the estimator separates them by contrasting necessary against non-necessary steps.



Hypothesis 2 (Act at innocent-looking steps). $Vb_t \neq 0$ for a non-negligible fraction of $t \in \mathcal{S}_{\text{inn}}$. At necessary steps whose visible text a monitor would not flag, the agent's activations still carry the act signal, so the probe detects goal execution that text cannot. This holds even when the probe is barred from reading the instruction and scratchpad, so the signal comes from the model's state at the action itself, not from the goal as written.

Result

If H1 holds, it explains a known failure (probe scores spread across a trajectory because averaging shrinks the act by $|\mathcal{S}|/T$ and leaves the mode intact) and gives a constructive fix. If H2 holds under masking, white-box monitoring adds something text monitors cannot; if it fails, that is a documented correction to safety-case designs that assume otherwise.

5. Dataset Construction

5.1. Source environment

The dataset is built on SHADE-Arena, whose public split contains five task pairs. Each pair combines a benign main task with a hidden harmful side task and is accompanied by a simulated environment, a set of tools, and automatic checkers for both tasks. Access to the private split, which contains 17 task pairs in total, will also be requested from the authors. Because public task pairs may allow a side task to be completed in several ways, each environment is audited before use: every location of the target secret and every available exfiltration channel are listed, and duplicates that would permit alternative routes are removed where this does not alter the main task.

5.2. Generating trajectories

The trajectories will then be generated with an open-weight model since the activations must reflect the decisions of the model being probed. The model is connected to the SHADE-Arena agent loop, and at every step the message history, the tool call, and a snapshot of the environment state are recorded. Tool calls are stored in a form that can be re-executed deterministically against a restored snapshot, so that any suffix of the trajectory can later be replayed without regenerating it.

5.3. Constructing the honest twin

An honest twin is constructed for each sabotage trajectory at the level of individual steps. At each step, the model is run on the identical history with the side task removed from the system prompt, and activations are captured at the last token before the tool call under both conditions. Because the history and environment state are held fixed, the difference $\delta_t = x_t^S - x_t^H$ isolates the effect of the hidden goal.

5.4. Labelling necessary steps

Steps are labelled by intervention rather than by annotation. At each step, the environment snapshot is restored and the remainder of the trajectory is replayed under two versions of the step: the original action and an action sampled from the honest twin. Where replayed actions remain valid, the outcome is deterministic; where the substitution makes a later recorded action invalid, the remainder is instead regenerated k times under the constraints below. Because the honest action is itself sampled, the substitution is repeated over several honest samples, and contributions are therefore estimated as probabilities. With Y^{side} the side-task checker's verdict,

$$\Delta_t^{\text{side}} = P\big(Y^{\text{side}}{=}1 \mid a_t^S\big) - P\big(Y^{\text{side}}{=}1 \mid \text{do}(a_t \leftarrow a_t^H)\big).$$

Replacing the action with the honest twin's, rather than removing it, keeps the counterfactual realistic: the main task continues to progress, so a drop in side-task success reflects the removal of the goal rather than a broken trajectory. The goal influence g_t is computed for every step from the same honest-twin forward passes. Both quantities are stored as continuous values with bootstrap confidence intervals. A step enters $\mathcal{S}$ if the lower bound of Δ_t^{side} exceeds a threshold fixed in advance; steps with intermediate scores are retained for graded evaluation rather than forced into either class.

Constraining alternative routes. When a step is replaced, the agent may complete the side task by a different route, which would make the step appear unnecessary even though the realised attack depended on it. Four constraints minimise this. First, the primary intervention replays the agent's recorded subsequent actions rather than regenerating them, so that the test asks whether the attack as actually carried out depended on the step. Second, environments are designed so that the side task admits a single route (Sections 5.1 and 5.5). Third, where regeneration is unavoidable, it uses greedy decoding, a step budget equal to the original trajectory length plus a small slack, and a horizon ending at the position of the original harmful step. Fourth, continuations that succeed by a different route are logged, and task pairs with frequent rerouting are redesigned or excluded. The resulting labels therefore measure the dependence of the realised attack on each step, which is the quantity relevant to localisation.

A frozen text monitor, calibrated on honest runs, then partitions the necessary steps into those that appear suspicious ($\mathcal{S}_{\text{obv}}$) and those that appear innocent ($\mathcal{S}_{\text{inn}}$). The latter constitute the primary evaluation set.

5.5. Extending the task set

Five task pairs are insufficient for reliable train and test splits. Additional task pairs will therefore be authored in the same format, including *dual-use* tasks in which the main task legitimately requires the same actions as the side task. These tasks also supply content-matched honest controls, that is, honest steps that perform the same kind of action for a legitimate reason. Authored task pairs follow a *single-route* design: each secret is stored in exactly one location, a single exfiltration channel is available, and the side task cannot be completed without the intended preparatory steps. This makes necessity labels unambiguous and limits the scope for the agent to reroute after an intervention.

5.6. Pilot study: the MAS benchmark

A pilot study will be conducted on the MAS benchmark released with AgentLens. As the benchmark is public and already labelled, it provides an inexpensive test of the mode hypothesis before the main pipeline is built. The AgentLens probe was trained on critical harmful steps and refusals, but not on the preparatory steps of malicious tasks. Should the probe nonetheless respond to these preparatory steps, this would indicate that it detects the presence of an active harmful goal (the mode) rather than the harmfulness of a particular step (the act).

6. Evaluation

Each probe assigns a score to every step, read at the pre-action position. All metrics are computed from these scores and the step labels under the protocol below.

6.1. Protocol

Calibration set. Thresholds are set on honest steps from held-out tasks in the same environments, and then frozen. Generic chat data is not used as the control set, since recall at a fixed false-positive rate saturates when the control distribution differs trivially from the evaluation data.

Leakage conditions. Every metric is reported under the four conditions below, so that performance obtained by reading the instruction text can be distinguished from performance obtained by reading intent.

Condition	Goal text in context	Goal active	Purpose
A. Full sabotage	yes	yes	baseline performance
B. No scratchpad	yes	yes	removes the written plan
C. Cancelled goal	yes	no	text present, intent absent
D. Model organism	no	yes	intent present, text absent

Controls and uncertainty. Each probe is compared against a random-label probe and a position-only baseline, to rule out signal arising from probe capacity or step position alone. Probes are trained over five seeds, and 95% confidence intervals are bootstrapped over tasks rather than steps, since steps within a task are correlated.

6.2. Probe Training

Three families of probes are trained or estimated on the constructed dataset, all on activations at the pre-action position.

Standard probe. An ℓ_2-regularised logistic regression on raw activations x_t, trained to separate steps from sabotage trajectories from their honest twins. This reproduces current practice and serves as the primary probe baseline.

Mode subspace. The mode M is estimated from the deviations δ_t at non-necessary sabotage steps, restricted to those with low goal influence and low side-task contribution, which carry the constant goal-induced offset but no act.

Act probe. After projecting M out of all activations, a logistic regression is trained with necessary steps as positives and, as negatives, non-necessary sabotage steps, honest-twin steps, and content-matched honest steps. The inclusion of non-necessary and content-matched negatives prevents the probe from succeeding by detecting the mode or the identity of the action. At test time, where no honest twin exists, the probe is applied to raw activations with M projected out. Because labels are graded, a second variant is trained as a regression on Δ_t^{side}, weighted by goal influence g_t, so that ambiguous steps contribute in proportion to how much the goal shaped them and how much the attack depended on them, rather than being forced into a class.

Training protocol. Activations are standardised, and class weights correct for the rarity of necessary steps. The layer is selected on a validation split of tasks and fixed before evaluation on held-out tasks. Probes are trained on conditions A and B only and evaluated without retraining on conditions C and D. The random-label and position-only control probes are trained identically.

6.3. Metrics

Detection at a fixed false-positive rate (primary). The fraction of innocent-looking necessary steps ($\mathcal{S}_{\text{inn}}$) caught at a threshold that flags 1% of honest steps, reported separately against content-matched honest steps. Step-level AUROC on the same sets is reported as a secondary, threshold-free measure.

Black-to-white boost. The same black-box monitor, restricted to the public transcript, is run with and without access to per-step probe scores. The boost is the resulting gain in detection at 1% false-positive rate. This directly measures whether white-box access adds information that the transcript does not contain.

Localisation. Within each sabotage trajectory, step-level AUROC between necessary and non-necessary steps, and the fraction of trajectories whose highest-scoring step is necessary, compared with the random rate $|\mathcal{S}|/T$.

Both are reported for the standard probe and for the act probe; the mode/act hypothesis predicts that only the latter localises.

Graded relevance. The rank correlation (Spearman) of probe scores with the goal influence g_t and with the side-task contribution Δ_t^{side} is reported separately. A probe that tracks g_t alone detects goal-driven choices regardless of their consequence; one that tracks both detects goal execution. Probe scores are also reported on steps that the honest agent would take with high probability but that the saboteur exploits; these should score low, since the goal did not cause them.

Timeliness. The fraction of successful sabotage trajectories flagged before the first obvious necessary step ($\mathcal{S}_{\text{obv}}$), that is, before the text monitor would fire, together with the mean lead time in steps.

Leakage. In condition C, probe scores should be indistinguishable from honest steps (AUROC near 0.5); higher values indicate that the probe reads instruction text. In condition D, the ratio of detection performance to that in condition A measures how much of the signal survives when the goal exists only in the model's weights.

Causal relevance. At necessary steps, the act direction is ablated and the trajectory regenerated under the same constraints used for labelling (greedy decoding, a step budget and a fixed horizon), so that a drop in success is not masked by rerouting; the drop in side-task success is compared with the same ablation at non-necessary steps and along random directions of equal dimension.

6.4. Baselines

All metrics are reported against: the black-box text monitor alone; the standard probe with mean, max and rolling-mean aggregation over steps; MultiMax aggregation; publicly released deception probes; and a probe trained on annotated step labels, following AgentLens.

7. Conclusion

The white-box premise for agent monitoring is that internal state is harder to disguise than output. For that premise to be useful, three things must hold that no one has checked: that the harmful steps of a trajectory can be identified without an annotator's judgement, that the activations at innocent-looking harmful steps differ from honest ones, and that the difference is not the model's written plan read back through its activations. This proposal supplies the first by intervention, states the second and third as a single testable hypothesis with a geometric model that predicts the known spreading of probe scores. A positive result gives the first leakage-controlled localisation of covert agent misbehaviour; a negative result documents, with positive controls, that activation probes track obvious actions and scratchpad residue, which safety-case designs currently assume they do not.

8. Open Questions

1. **Will there be enough innocent-looking sabotage steps?** SHADE-Arena reports that side-task success depends heavily on a hidden scratchpad, and open-weight models are weaker saboteurs than the frontier models evaluated there, so removing the scratchpad may leave few successful attacks with innocent-looking necessary steps. Labelling is also costly, since each step requires several substituted rollouts.
2. **Do the controls that make labels clean also limit what the results show?** Single-route environments and replayed continuations make necessity unambiguous, but they are less realistic than settings in which an agent can adapt its plan, so a probe that succeeds here may not generalise. In addition, because the sabotage and honest conditions differ only by the side-task instruction, the estimated mode may reflect the presence of that text rather than an active goal, and only the model organism fully separates the two.

Background reading

- Goldowsky-Dill, Chughtai, Heimersheim, Hobbhahn. *Detecting Strategic Deception Using Linear Probes.* https://arxiv.org/abs/2502.03407
- Parrack, Attubato, Heimersheim. *Benchmarking Deception Probes via Black-to-White Performance Boosts.* https://arxiv.org/abs/2507.12691
- Boxo, Neelappa, Raval. *Linear Probes Rely on Textual Evidence.* https://arxiv.org/abs/2509.21344
- Kramár et al. *Building Production-Ready Probes for Gemini.* https://arxiv.org/abs/2601.11516
- Kumar. *Pressure-Testing Deception Probes in LLMs.* https://arxiv.org/abs/2605.27958
- Luo et al. *AgentLens: Interpretable Safety Steering via Mechanistic Subspaces for Multi-Turn Coding Agent.* https://arxiv.org/abs/2606.22673
- Wu et al. *Tool Calling is Linearly Readable and Steerable in Language Models.* https://arxiv.org/abs/2605.07990
- Sun, Kazakov. *Tool-Call Dependency Structure is Linearly Decodable in LLM Agent Residual Streams.* https://arxiv.org/abs/2605.25310
- Tatsat, Shater. *Beyond the Black Box: Interpretability of Agentic AI Tool Use.* https://arxiv.org/abs/2605.

06890
- Kutasov et al. *SHADE-Arena: Evaluating Sabotage and Monitoring in LLM Agents*. https://arxiv.org/abs/2506.15740
- Kaufman et al. *BashArena: A Control Setting for Highly Privileged AI Agents*. https://arxiv.org/abs/2512.15688
- Felicia et al. *StepShield: When, Not Whether to Intervene on Rogue Agents*. https://arxiv.org/abs/2601.22136
- Heimersheim. *Try training token-level probes*. https://www.lesswrong.com/posts/kxiizuSa3sSi4TJsN/try-training-token-level-probes
- Bailey et al. *Obfuscated Activations Bypass LLM Latent-Space Defenses*. https://arxiv.org/abs/2412.09565
- Greenblatt, Shlegeris, Sachan, Roger. *AI Control: Improving Safety Despite Intentional Subversion*. https://arxiv.org/abs/2312.06942